FlightLens: System Analysis & Design

DEPI Round 5, Phase 4: System Analysis & Design

This document follows the six parts of Phase 4 in the DEPI Project Documentation Guidelines: problem statement and objectives; database design and data modeling; data flow and system behavior; UI/UX design and prototyping; system deployment and integration; and additional deliverables. Use cases (UC), functional requirements (FR) and non-functional requirements (NFR) come from the Requirements Specification (Phase 3).

1. Problem statement and objectives

1.1 Problem statement

BTS publishes every US domestic flight, but in a form analysts cannot use directly. Data arrives as one large file per month. Times are local hhmm numbers, delay-cause fields are blank for most rows, and codes need lookup tables. Every analyst repeats the same cleaning and gets slightly different answers.

FlightLens cleans the data once, tests it, and publishes it as one trusted source.

1.2 Objectives

#	Business question	Delivered by
Q1	Which airports have the most delays and cancellations?	Dashboard
Q2	What causes delays: carrier, weather, air traffic, security or a late incoming aircraft?	Dashboard
Q3	Do delays rise in certain months, weekdays or hours?	Dashboard
Q4	Which carriers have the highest delay and cancellation rates?	Dashboard
Q5	Which routes are delayed most often?	Dashboard
Q6	Are long-distance flights more likely to be delayed?	Dashboard
Q7	Will a given scheduled flight arrive 15+ minutes late?	ML model
Q8	Can a manager get these answers by asking in plain English?	GenAI assistant

Tier	Features
MVP	Automated monthly download; Bronze, Silver and Gold layers; data-quality checks; star schema in PostgreSQL; Power BI dashboard (Q1–Q6); Airflow orchestration; Docker setup
Planned	Delay-prediction model with API (Q7); GenAI assistant (Q8); CI/CD with GitHub Actions
Stretch	Hourly weather enrichment; simulated real-time stream with Kafka

Scope: US domestic flights reported to BTS, July to December 2025 (3,554,943 flights).

1.3 Use case diagram and descriptions

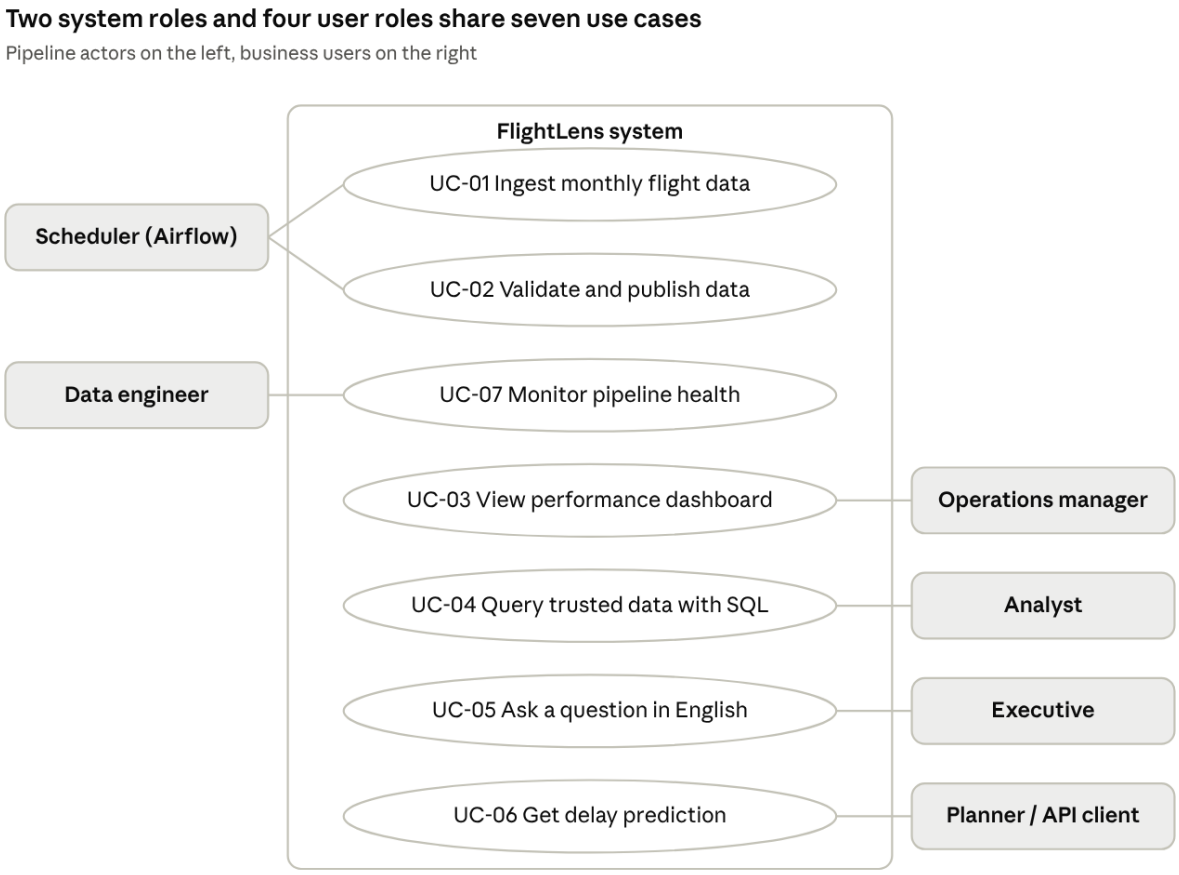


Figure 1. Use case diagram

ID	Use case	Actor	Main flow	Alternative / failure flow
UC-01	Ingest monthly flight data	Scheduler (Airflow); data engineer for backfill	Download the BTS file for the month, then store it unchanged in Bronze with load time and file name	If the download fails, it is retried 3 times; then the run is marked failed and an alert is sent
UC-02	Validate and publish data	Scheduler	Clean to Silver, run quality checks, build Gold tables, then refresh warehouse views	If a critical check fails, the rows go to quarantine, the run stops and an alert is sent
UC-03	View performance dashboard	Airline / airport operations manager	Pick date range, carrier and airport, then see KPIs, maps and rankings	If no data matches the filter, the page shows "no flights" instead of zeros
UC-04	Query trusted data with SQL	Analyst	Connect to the warehouse with a read-only user, then query Gold views using the data dictionary	A query on a raw layer is denied (read access to Gold only)
UC-05	Ask a question in plain English	Executive	The assistant sends the question and the schema to the LLM (Ollama or Gemini), gets SQL, runs it read-only, then shows the answer, table and SQL	If the SQL is invalid or not a SELECT, it is not run and the assistant asks to rephrase
UC-06	Get delay prediction	Planner (via app or API call)	The API builds the features, then the model returns the delay probability and an on-time or delayed label	An unknown airport or carrier returns a 400 error with a clear message
UC-07	Monitor pipeline health	Data engineer	Open the data-health page, see the last load, row counts and check results, then fix and re-run the month	Re-run replaces that month only (no duplicates)

1.4 Functional and non-functional requirements

ID	Functional requirement	Use case	Priority
FR-01	Download BTS on-time data for a given month automatically and on a monthly schedule, plus a backfill of July–December 2025	UC-01	Must
FR-02	Store every raw file unchanged in the Bronze layer with load metadata	UC-01	Must
FR-03	Clean, type and standardise records into Silver (times, nulls, flags, lookups)	UC-02	Must
FR-04	Run data-quality checks on each load and quarantine failing rows	UC-02	Must
FR-05	Build the Gold star schema and daily KPI aggregates	UC-02	Must
FR-06	Join hourly weather to flights by airport and hour	UC-02	Could
FR-07	Expose Gold tables through a SQL endpoint with read-only views	UC-04	Must
FR-08	Provide a Power BI dashboard answering Q1–Q7 with date, carrier, airport and state filters	UC-03	Must
FR-09	Train, evaluate and register a delay-prediction model; serve predictions through a REST API	UC-06	Should
FR-10	Answer plain-English questions by generating and running read-only SQL, showing the SQL used	UC-05	Should
FR-11	Show pipeline status and data-quality results on a data-health page	UC-07	Must

ID	Category	Non-functional requirement
NFR-01	Performance	Monthly incremental load under 30 minutes; dashboard pages load in under 5 seconds; prediction API under 500 ms; assistant answer under 10 seconds
NFR-02	Scalability	Handles July–December 2025 (3,554,943 rows) and grows month by month without redesign (partitioning by year and month)
NFR-03	Reliability	Re-running a month gives the same result (idempotent); failed runs retry and alert; 95% of scheduled runs succeed
NFR-04	Data quality	100% of critical checks pass before data reaches Gold
NFR-05	Security	Secrets in an .env file kept out of Git; role-based access; assistant uses a read-only SQL user
NFR-06	Usability	Dashboard usable without training; assistant answers in plain language
NFR-07	Maintainability	Modular, tested Python code; one-command Docker Compose setup; README and data dictionary
NFR-08	Cost	Runs on team laptops with free, open-source software; no paid cloud services

1.5 Software architecture

Architecture style: a layered (medallion) data pipeline, with each service in its own Docker container. Airflow lands raw files, PySpark refines them through three Delta Lake layers, and PostgreSQL serves the Gold layer to every consumer. Each hop between layers is gated by quality checks. A month is loaded once; re-running it replaces that month instead of adding duplicates.

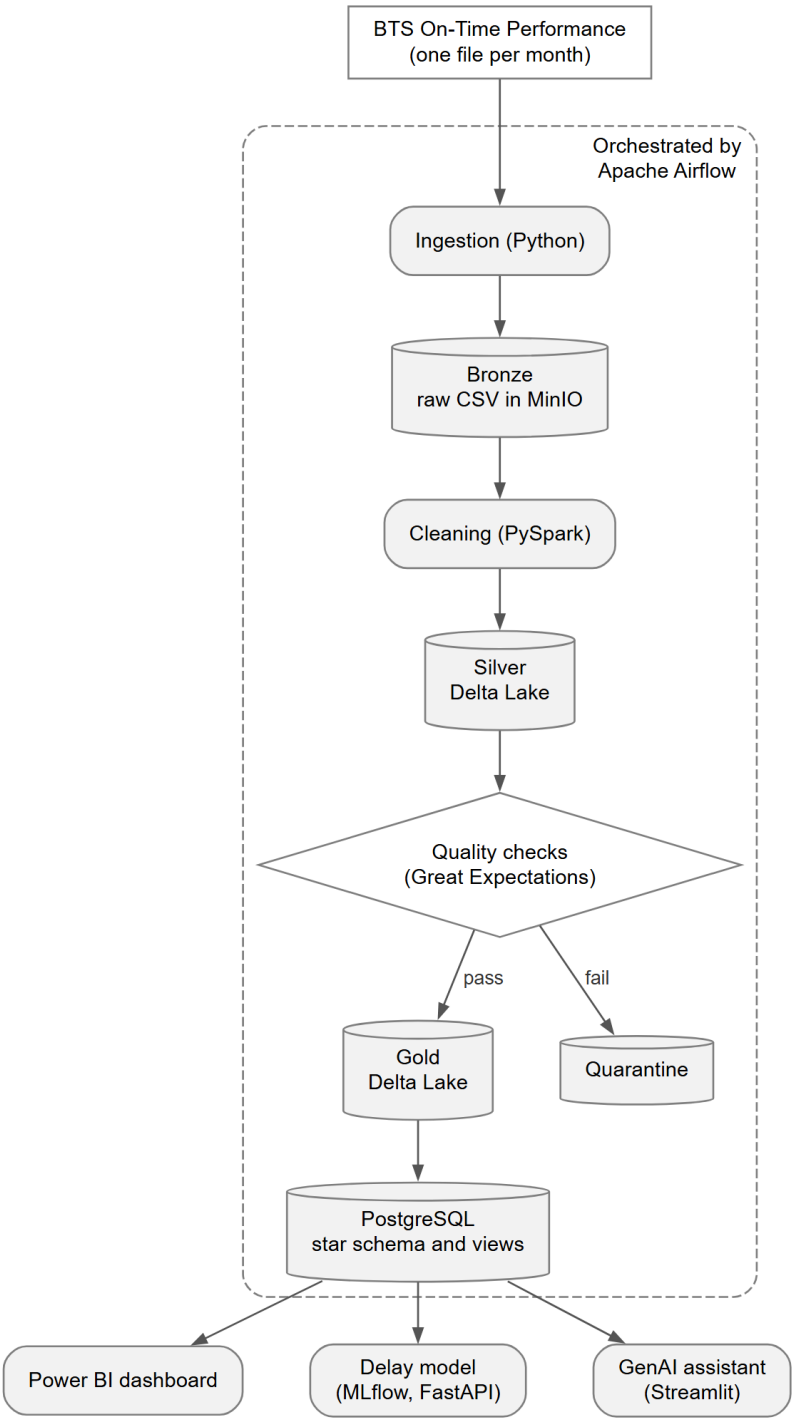


Figure 2. Software architecture

2. Database design and data modeling

The Gold layer is a star schema in PostgreSQL 16 with one row per scheduled flight in the fact table, so every business question becomes a simple join and group-by.

2.1 ER diagram

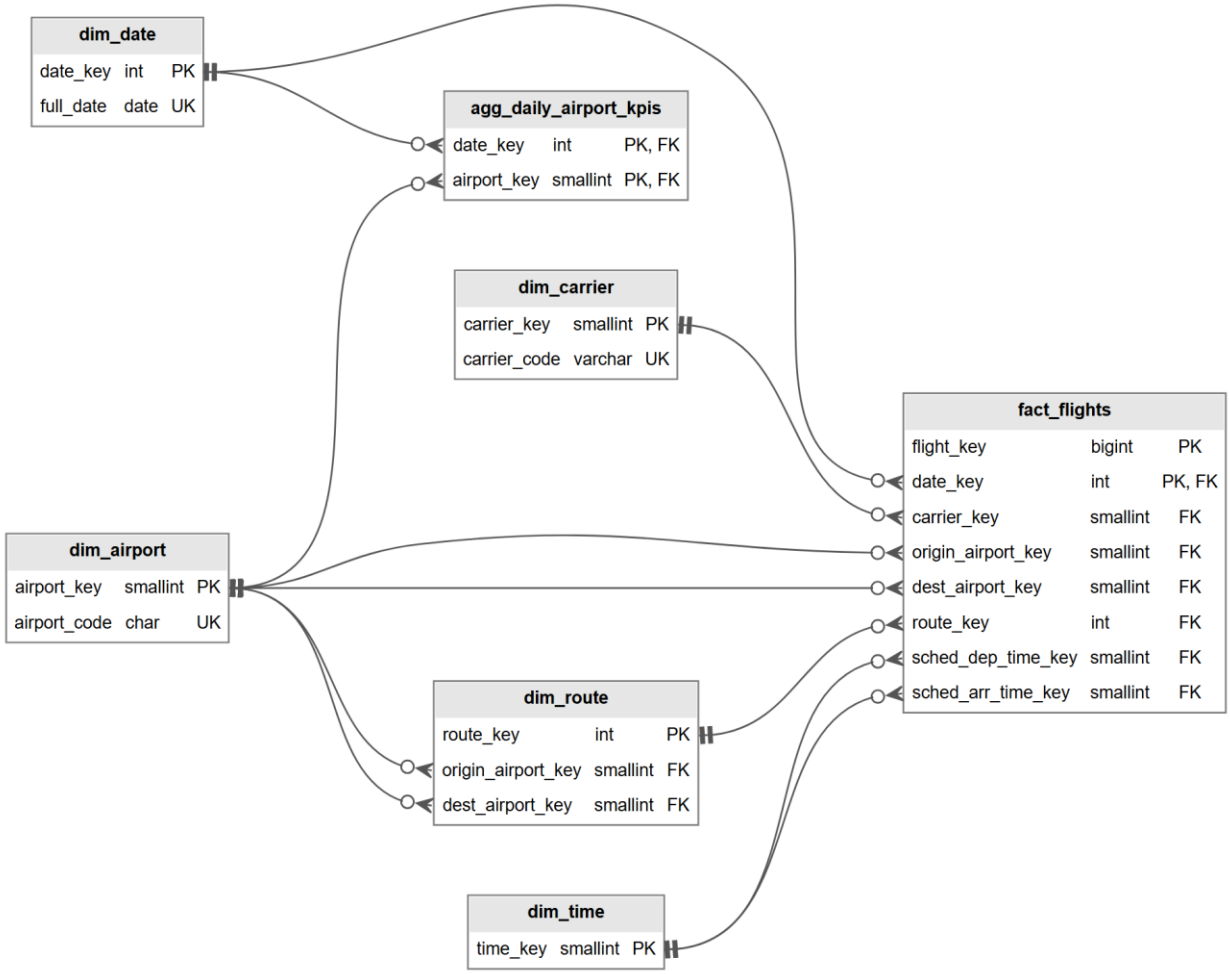


Figure 3. ER diagram of the Gold star schema

dim_airport is a role-playing dimension: the fact table points to it twice, once as the origin and once as the destination. The table load_audit (section 2.2) stores the data-quality results of each load and has no link to the star schema.

2.2 Logical schema

Table	Type	One row per	Main attributes
fact_flights	Fact	scheduled flight	Keys to every dimension; flight and tail number; scheduled and actual times; delays, taxi and air time; 15-minute flags; cancellation and code; diversion; flight status; five delay causes in minutes; load month
dim_date	Dimension	day (2025–2026)	Year, quarter, month, weekday, weekend flag, US federal holiday flag and name
dim_time	Dimension	hour of day (24 rows)	Hour label, BTS time block, day part (Night, Morning, Afternoon, Evening)
dim_carrier	Dimension	airline	Carrier code and name
dim_airport	Dimension	airport	Airport code, name, city, state, latitude and longitude (for the map page)
dim_route	Dimension	origin and destination pair	Route code, distance in miles, distance group and its sort order
agg_daily_airport_kpis	Aggregate	day and airport	Departures, arrivals, delayed, cancelled and diverted flights, average delays, delay minutes by cause
load_audit	Audit	month and quality check	Check name, value, expected value, passed flag, check time (data-health page, FR-11)

#	Question	Tables
Q1	Airports with most delays and cancellations	agg_daily_airport_kpis, dim_airport, fact_flights (cancellation code)
Q2	Causes of delay	fact_flights delay-cause columns
Q3	Delays by month, weekday and hour	dim_date, dim_time
Q4	Carriers' delay and cancellation rates	dim_carrier
Q5	Most delayed routes	dim_route
Q6	Distance and delay	dim_route distance group
Q7	Delay prediction (ML)	fact_flights and dimensions, features known before departure only
Q8	Questions in plain English	all read-only views, with this schema as the data dictionary

2.3 Physical schema

Tables and columns

fact_flights

Column	Type	Key	Description
flight_key	BIGINT	PK	Surrogate key (IDENTITY)
date_key	INTEGER	PK, FK to dim_date	Flight date, also the partition key
carrier_key	SMALLINT	FK to dim_carrier	Operating carrier
origin_airport_key	SMALLINT	FK to dim_airport	Departure airport
dest_airport_key	SMALLINT	FK to dim_airport	Arrival airport
route_key	INTEGER	FK to dim_route	Origin and destination pair
sched_dep_time_key	SMALLINT	FK to dim_time	Scheduled departure hour
sched_arr_time_key	SMALLINT	FK to dim_time	Scheduled arrival hour
flight_number	SMALLINT		Carrier flight number
tail_number	VARCHAR(10)		Aircraft registration; can be empty
sched_departure, actual_departure	TIMESTAMP		Local departure times
sched_arrival, actual_arrival	TIMESTAMP		Local arrival times; can be on the next day
dep_delay_min, arr_delay_min	SMALLINT		Delay in minutes; negative when early
is_dep_delayed_15, is_arr_delayed_15	BOOLEAN		15 minutes or more late
taxi_out_min, taxi_in_min	SMALLINT		Taxi time in minutes
sched_elapsed_min, actual_elapsed_min, air_time_min	SMALLINT		Flight durations in minutes
is_cancelled, is_diverted	BOOLEAN		Flight outcome

Column	Type	Key	Description
cancellation_code	CHAR(1)		A carrier, B weather, C National Air System, D security
flight_status	VARCHAR(9)		On time, Delayed, Cancelled or Diverted
carrier_delay_min, weather_delay_min, nas_delay_min, security_delay_min, late_aircraft_delay_min	SMALLINT		Delay minutes by cause; 0 for flights less than 15 minutes late, empty for cancelled and diverted flights
load_month	CHAR(7)		Month of the load, for example 2025-07
loaded_at	TIMESTAMPTZ		Load time

dim_date

Column	Type	Key	Description
date_key	INTEGER	PK	Smart key, for example 20250704
full_date	DATE	UNIQUE	Calendar date
year, quarter, month, day_of_month, day_of_week	SMALLINT		Date parts
month_name, day_name	VARCHAR		Names for labels
year_month	CHAR(7)		For example 2025-07
is_weekend, is_holiday	BOOLEAN		Weekend and US federal holiday flags
holiday_name	VARCHAR		Name of the holiday, if any

dim_time

Column	Type	Key	Description
time_key	SMALLINT	PK	Hour of day, 0–23
hour_label	CHAR		For example 06:00
bts_time_block	CHAR		BTS hour block, for example 0600-0659
day_part	VARCHAR		Night, Morning, Afternoon or Evening

dim_carrier

Column	Type	Key	Description
carrier_key	SMALLINT	PK	Surrogate key
carrier_code	VARCHAR(3)	UNIQUE	IATA code, for example AA
carrier_name	VARCHAR		Carrier name

dim_airport

Column	Type	Key	Description
airport_key	SMALLINT	PK	Surrogate key
airport_code	CHAR(3)	UNIQUE	IATA code, for example ATL
airport_name, city	VARCHAR		From the BTS airport lookup
state_code	CHAR(2)		State or territory
state_name	VARCHAR		State name
latitude, longitude	NUMERIC		For the map page

dim_route

Column	Type	Key	Description
route_key	INTEGER	PK	Surrogate key
origin_airport_key, dest_airport_key	SMALLINT	FK to dim_airport	Both ends of the route
route_code	CHAR		For example ATL-LAX
distance_miles	SMALLINT		Route distance
distance_group	VARCHAR		Under 500, 500-999, 1000-1499, 1500-1999, 2000 and over
distance_group_order	SMALLINT		Sort order of the distance group

agg_daily_airport_kpis

Column	Type	Key	Description
date_key	INTEGER	PK, FK to dim_date	Day
airport_key	SMALLINT	PK, FK to dim_airport	Airport
departures, departures_delayed_15, cancelled, diverted	INTEGER		Departures from the airport
avg_dep_delay_min	NUMERIC		Average departure delay
arrivals, arrivals_delayed_15	INTEGER		Arrivals at the airport
avg_arr_delay_min	NUMERIC		Average arrival delay
carrier_delay_min … late_aircraft_delay_min	INTEGER		Delay minutes by cause for departures

load_audit

Column	Type	Key	Description
load_month	CHAR(7)	PK	Month of the load
check_name	VARCHAR	PK	Quality check
check_value	BIGINT		Result of the check
expected_value	BIGINT		Expected result; empty when for information only
passed	BOOLEAN		Computed: no expected value, or value = expected
checked_at	TIMESTAMPTZ		Check time

Keys

- Surrogate keys (GENERATED BY DEFAULT AS IDENTITY) on carrier, airport and route, so codes can change without breaking the fact table.
- Smart keys on date (20250704) and time (hour 0–23), because they are stable and readable.
- The natural key of a flight is protected by UNIQUE (date_key, carrier_key, flight_number, origin_airport_key, dest_airport_key, sched_departure).
- All fact foreign keys are enforced with REFERENCES.

Partitioning

fact_flights is partitioned by RANGE (date_key), one partition per month (fact_flights_2025_07 to fact_flights_2025_12). Month loads stay fast, a re-run replaces one month only, and queries skip the months they do not need (NFR-02). A new month needs one new partition.

Indexes

Index	Columns	Why
Primary key	flight_key, date_key	A partitioned table needs the partition key in the primary key
Unique	natural key of the flight	No duplicate flights
ix_fact_flights_carrier	carrier_key, date_key	Carriers page, filtered by date
ix_fact_flights_origin	origin_airport_key, date_key	Airports page, filtered by date
ix_fact_flights_dest	dest_airport_key, date_key	Arrivals per airport
ix_fact_flights_route	route_key	Routes page
ix_fact_flights_dep_hour	sched_dep_time_key	Hour by weekday heatmap
ix_fact_flights_status	flight_status	On-time / delayed / cancelled splits

Data rules enforced by the database

- A cancelled flight always has a cancellation code (A carrier, B weather, C National Air System, D security), and only cancelled flights have one.
- A flight cannot be both cancelled and diverted.
- flight_status is one of On time, Delayed, Cancelled, Diverted.
- No negative taxi times, flight times or delay-cause minutes; the scheduled flight time is more than 0.
- Rows that break a rule go to quarantine before the load (FR-04).

Type choices

Data	Type	Reason
Minutes and miles	SMALLINT	Max delay 4,352 minutes and max distance 5,095 miles fit; half the size of INTEGER on 3.5M rows
Flags	BOOLEAN	BTS sends 0.0 / 1.0; booleans are clearer in SQL and Power BI
Times	TIMESTAMP (no time zone)	BTS times are local to each airport; durations come from the elapsed-time columns
Codes	CHAR(3) airports, VARCHAR(3) carriers	Fixed IATA codes
Flight key	BIGINT	3,554,943 rows and growing every month

Read-only views and access

View	Used by	Content
vw_flights	Dashboard, assistant, ML	One row per flight with all names
vw_carrier_monthly	Carriers page (Q4)	On-time %, cancellation %, average delay per carrier and month
vw_route_performance	Routes and distance page (Q5, Q6)	Delay % per route with distance group
vw_delay_causes	Delay causes page (Q2)	Delay minutes by cause, date, carrier and airport
vw_airport_daily	Airports page and map (Q1)	Daily KPIs per airport with location

The role flightlens_reader can read these views and nothing else (FR-07, NFR-05). The dashboard, the assistant and the analysts log in with users that have this role.

2.4 Normalization

- Fact table: every measure depends on the whole flight and nothing else. Names and descriptions are not stored in the fact table; it holds only keys to the dimensions, so a carrier or airport name is stored once.
- Dimensions: denormalized on purpose, as a star schema and not a snowflake. For example, dim_airport keeps the city and the state in the same row, and dim_date keeps the month name, quarter and weekday, instead of separate city, state or month tables. This means one join per dimension for Power BI, analysts and the assistant's SQL. The repetition costs little, because the dimensions are small (14 carriers, 351 airports, 6,695 routes).
- Route distance depends on the route only, so it is stored in dim_route, not repeated in every flight.
- Aggregate table: agg_daily_airport_kpis is a planned copy of numbers that can be computed from the fact table, kept for speed. It is rebuilt for the month after every load, so it never drifts from the fact table.

3. Data flow and system behavior

3.1 Data flow diagram

Context level (level 0)

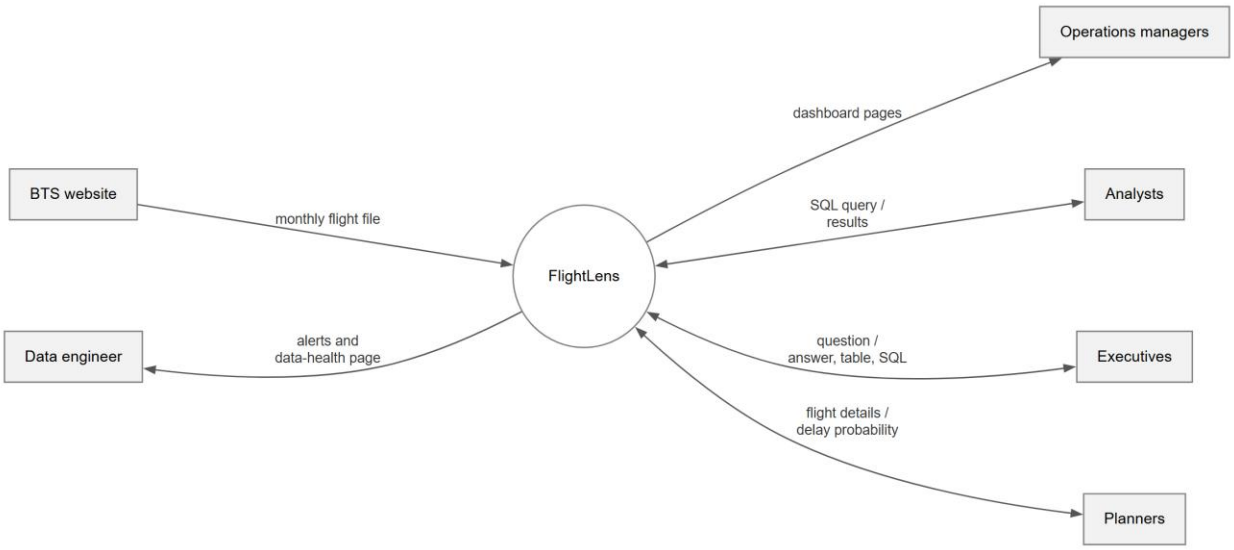


Figure 4. Context-level data flow diagram

Detailed level (level 1)

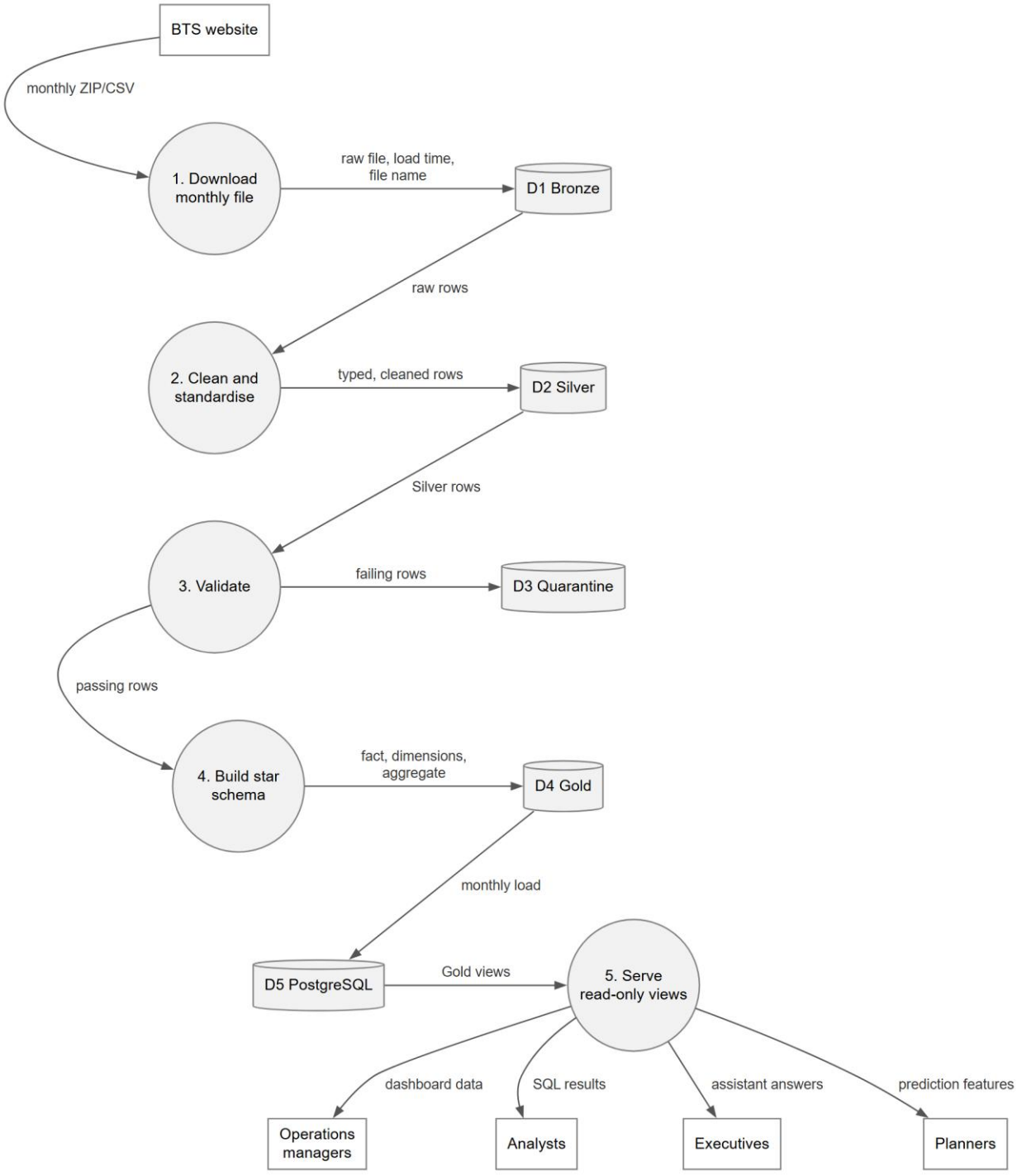


Figure 5. Level-1 data flow diagram

Store	Content
D1 Bronze	Every raw file unchanged, with load metadata (FR-02)
D2 Silver	Cleaned and typed records: times, nulls, flags, lookups (FR-03)
D3 Quarantine	Rows that fail a critical check (FR-04)
D4 Gold	Star schema and daily KPI aggregates (FR-05)
D5 PostgreSQL	Gold tables and read-only views (FR-07)

3.2 Sequence diagrams

Prediction request (UC-06)

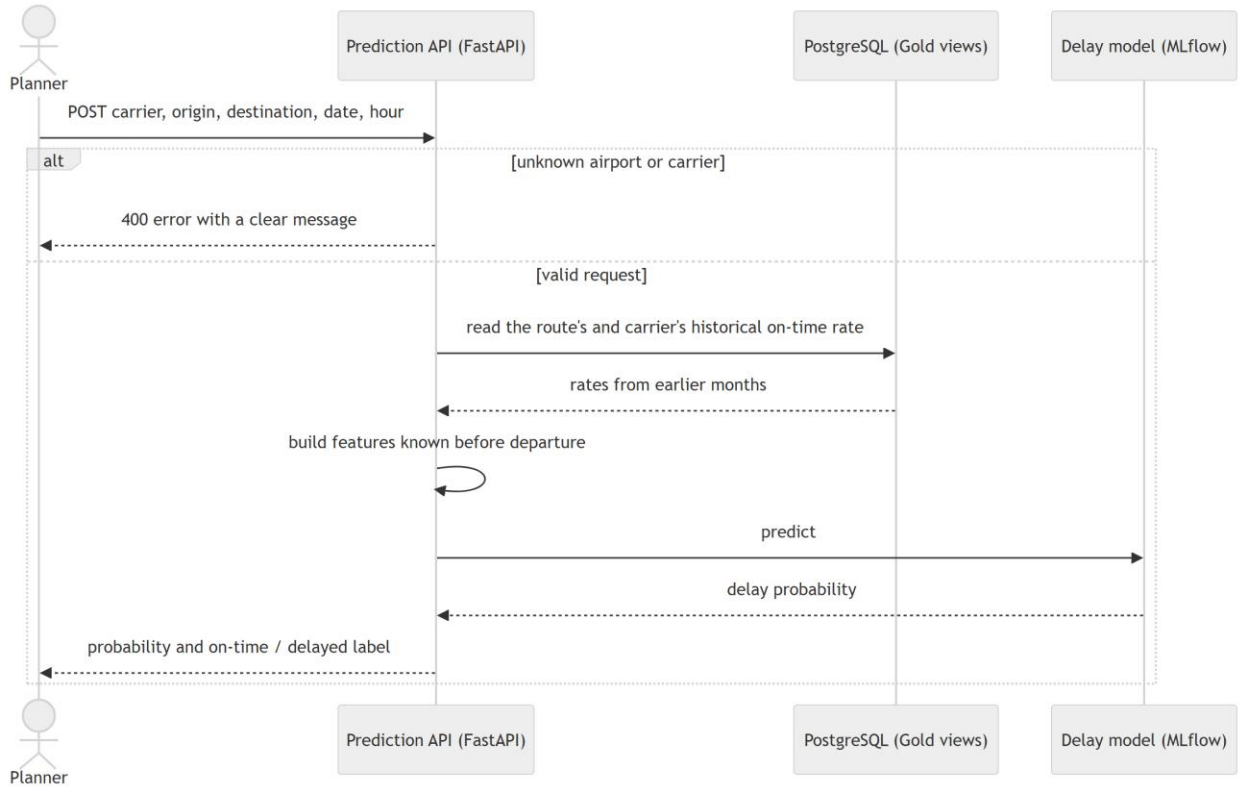


Figure 6. Sequence diagram: prediction request

Assistant question (UC-05)

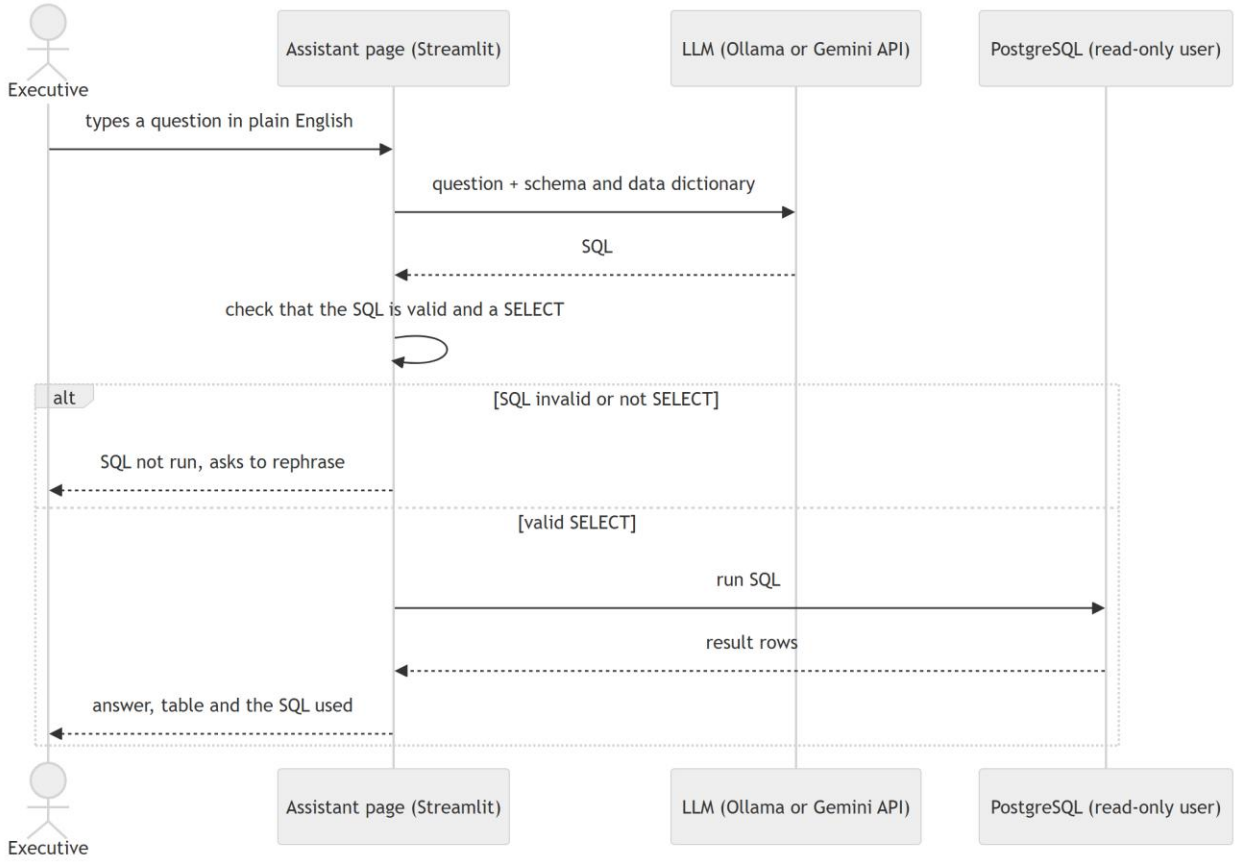


Figure 7. Sequence diagram: assistant question

3.3 Activity diagram

One monthly pipeline run, with the quality gate and the quarantine branch (UC-01, UC-02).

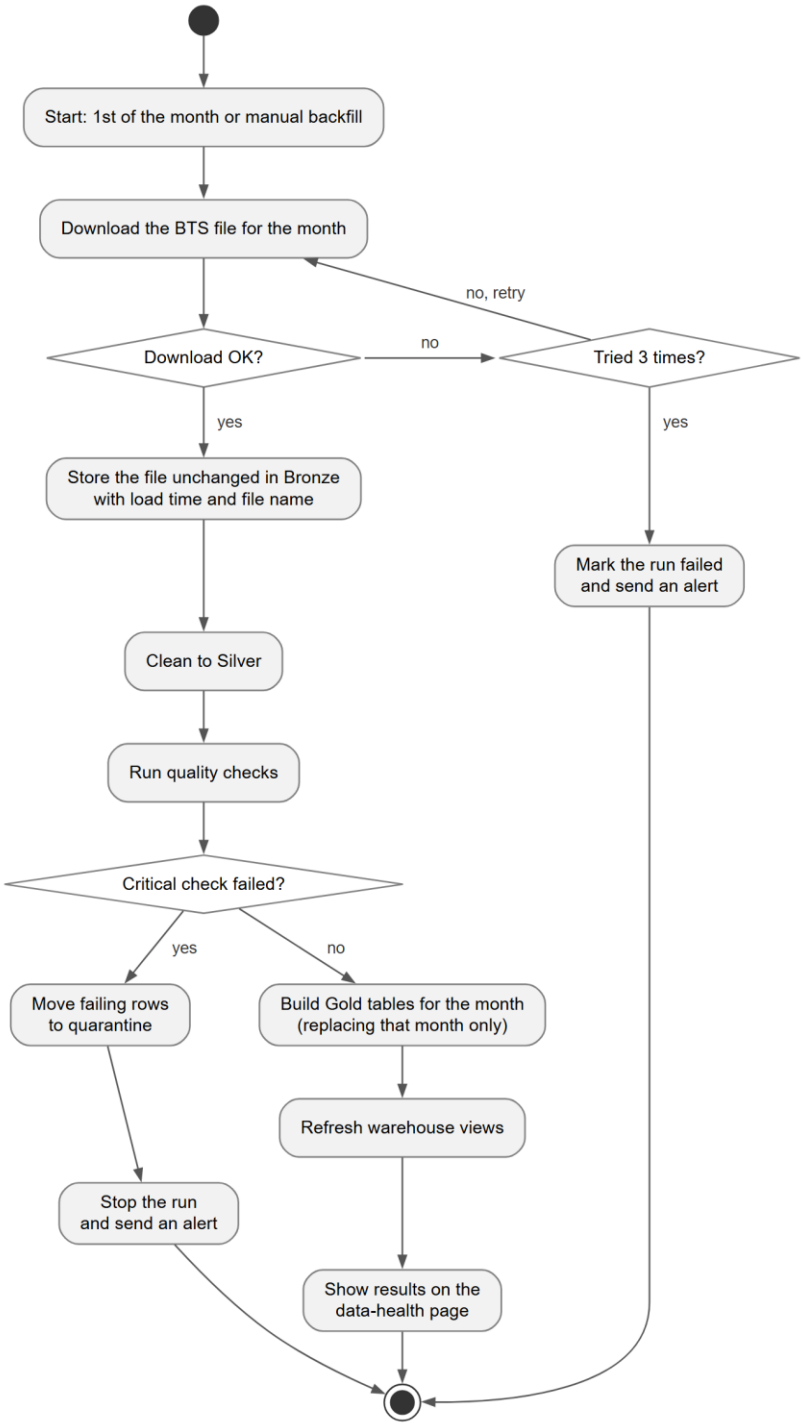


Figure 8. Activity diagram: monthly pipeline run

3.4 State diagrams

Pipeline run

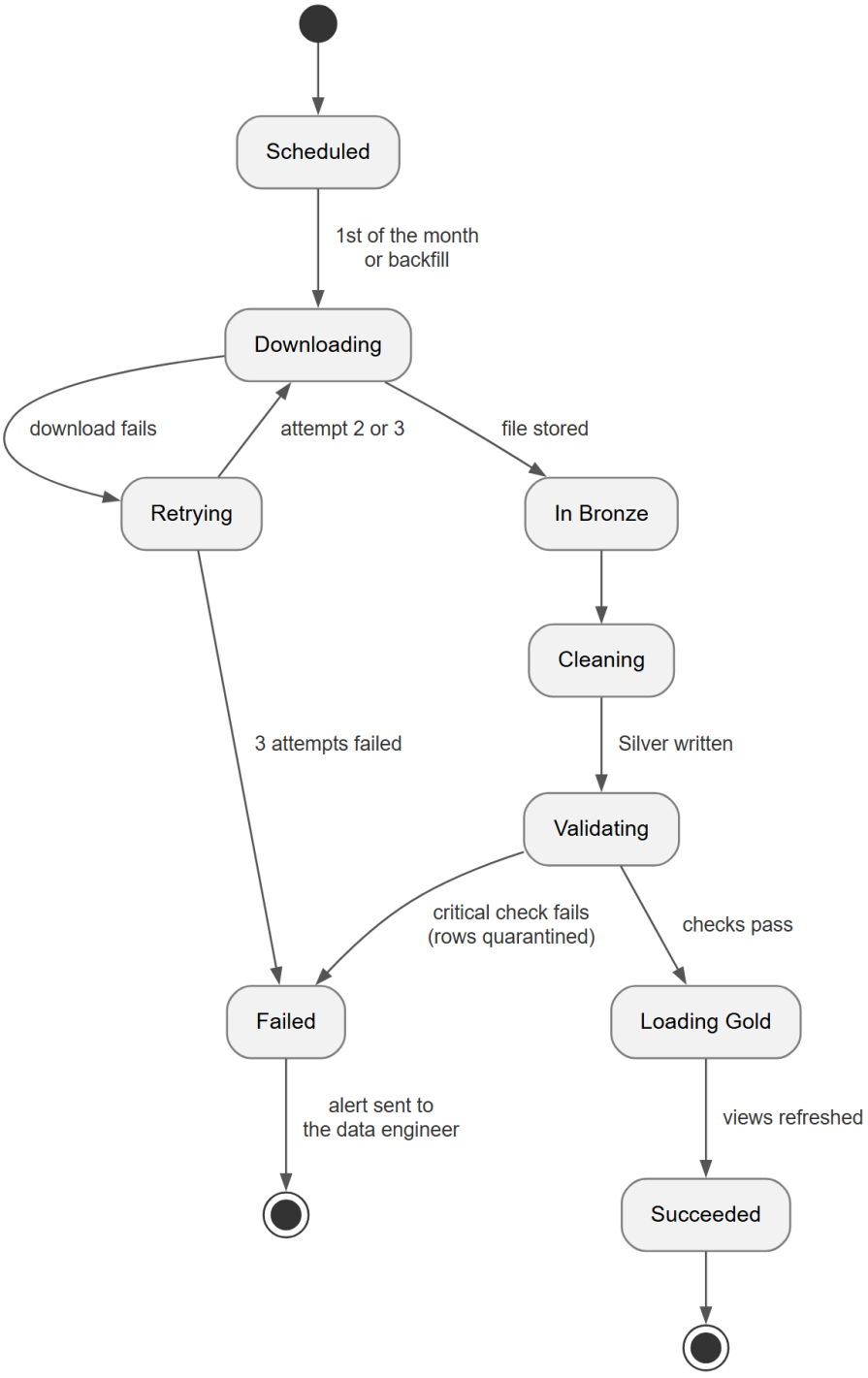


Figure 9. State diagram: pipeline run

When a run fails, the data engineer fixes the cause and re-runs the month, which starts again at Scheduled. A re-run replaces that month only, so the result is the same with no duplicates (NFR-03).

Flight status

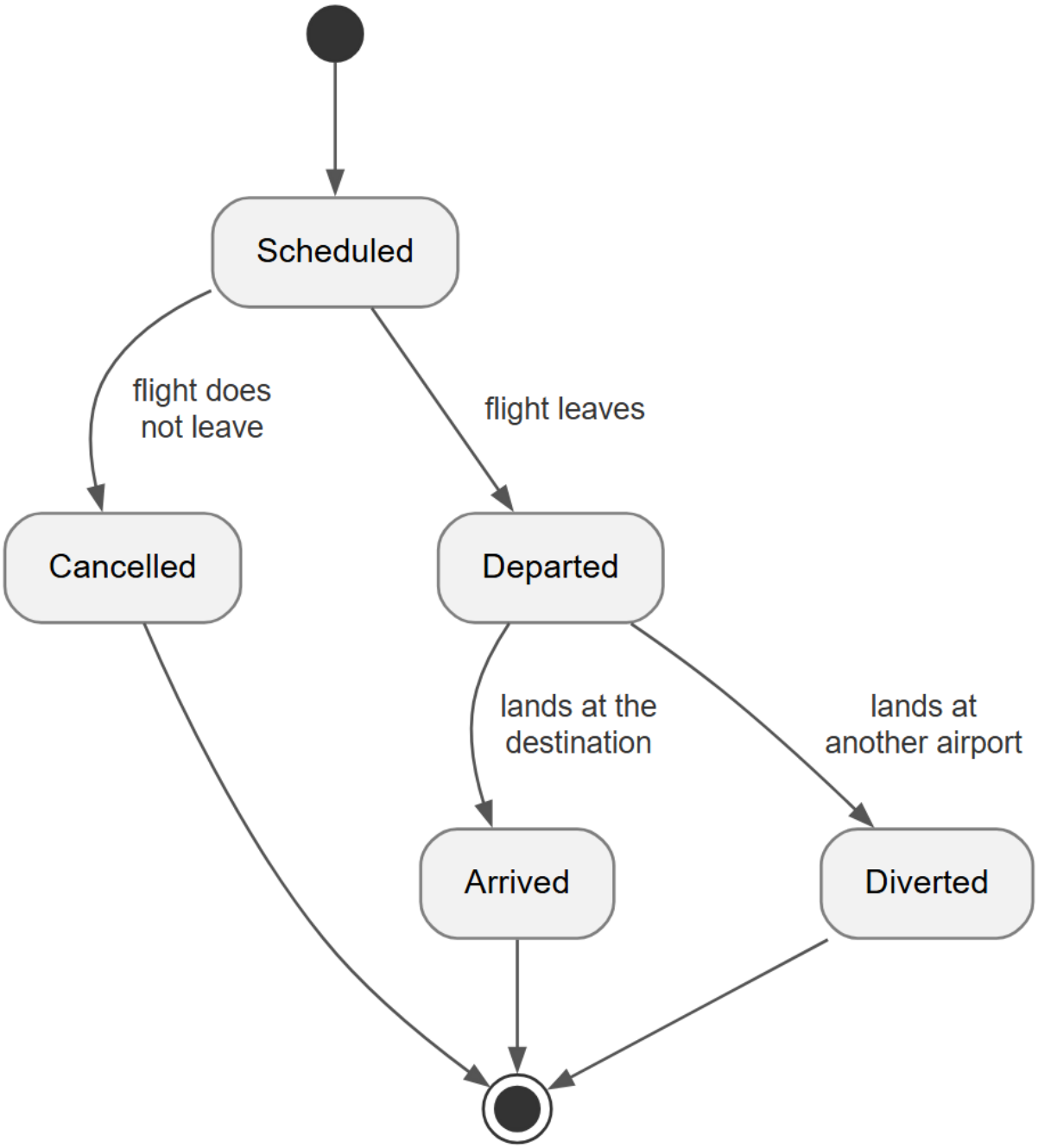


Figure 10. State diagram: flight status

In the Gold layer an arrived flight is stored as On time (less than 15 minutes late) or Delayed (15 minutes or more), so flight_status holds On time, Delayed, Cancelled or Diverted.

3.5 Class diagram

The Python package of the pipeline: downloader, transformer, validator and loader. Each Airflow task calls one class for one month.

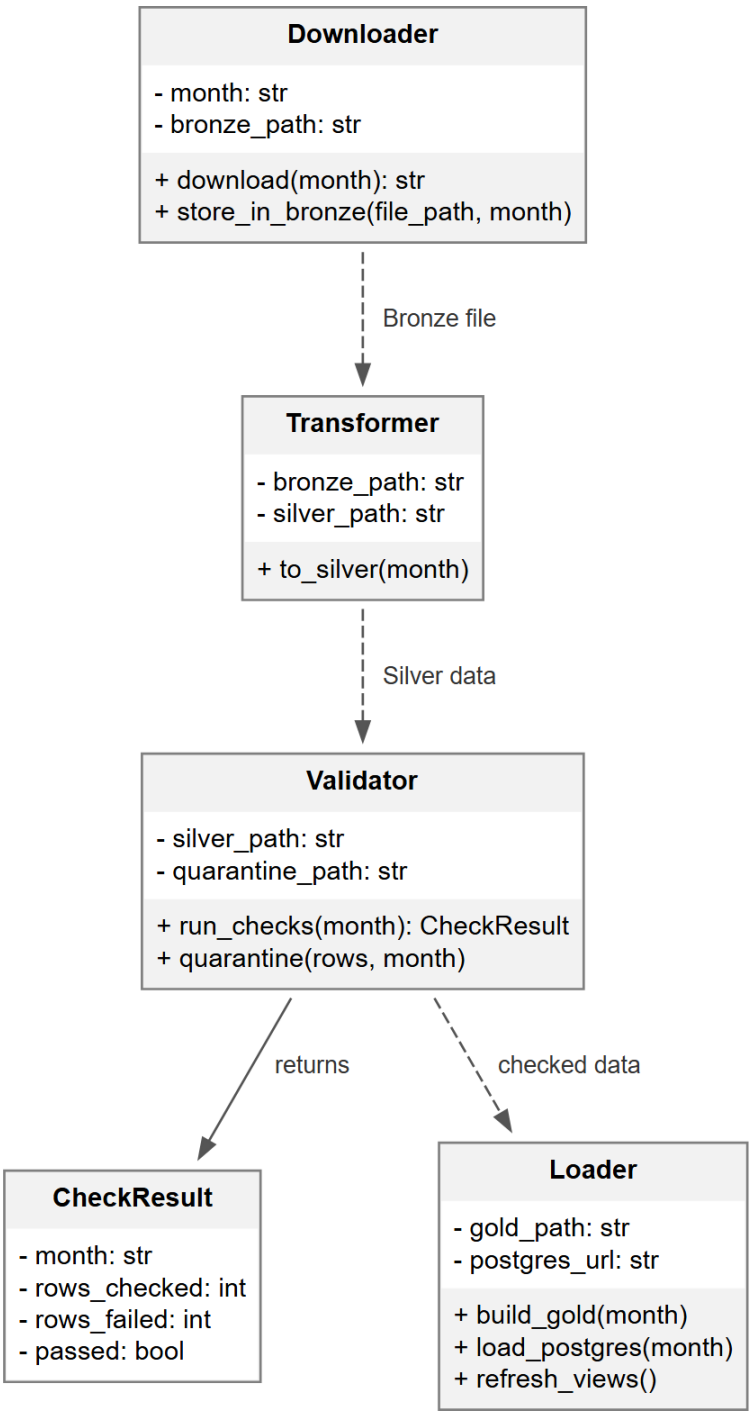


Figure 11. Class diagram of the pipeline package

4. UI/UX design and prototyping

4.1 Wireframes and mockups

The Power BI report has 7 pages, and the GenAI assistant has its own chat page. Every report page has the same frame: page navigation on the left, filters in the top right (date range, carrier, airport and state, as FR-08 requires; the Data health page filters by load month), a row of KPI cards, and four charts. The dark tag on each chart names the business question it answers, and the grey text at the bottom names its source.

Page 1: Overview (Q3)

Flights, on-time %, cancellation %, average delay and the monthly trend, with a short view of Q1, Q2 and Q4.

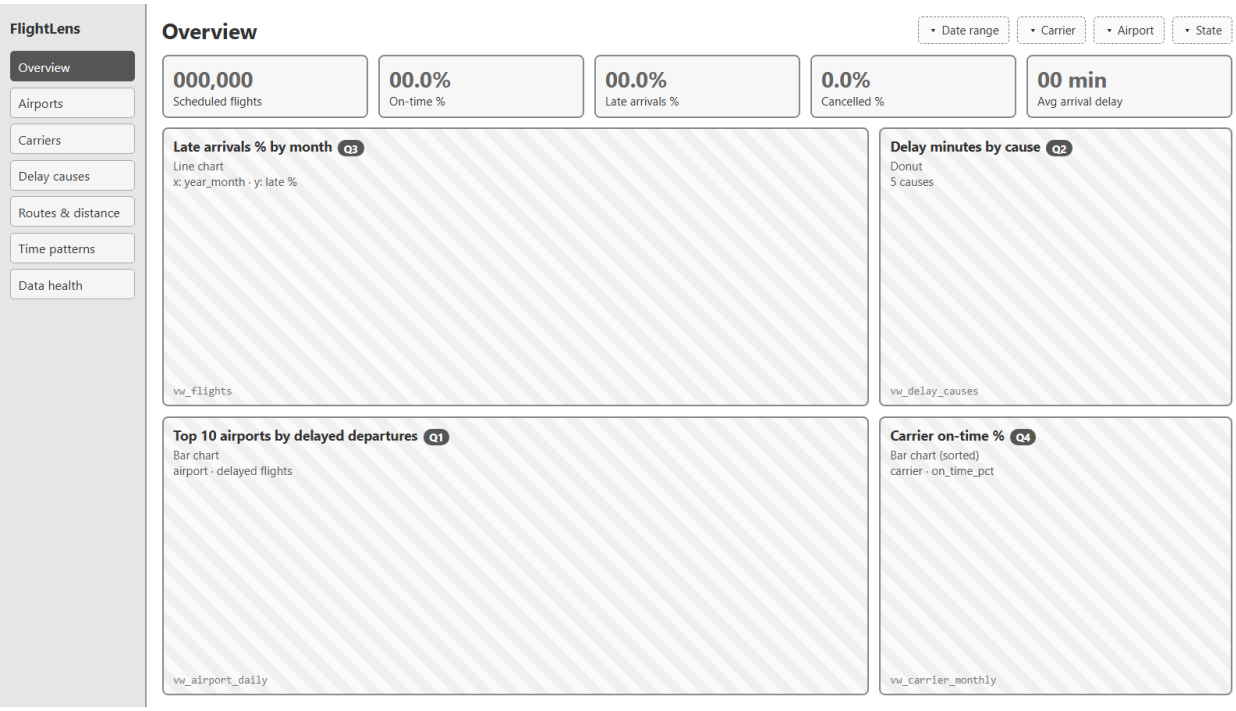


Figure 12. Wireframe: Overview (Q3)

Page 2: Airports (Q1)

A US map of airports by on-time %, the 10 worst airports, the daily trend for the selected airport and cancellations by reason (Carrier, Weather, NAS, Security). Sources: vw_airport_daily and vw_flights.



Figure 13. Wireframe: Airports (Q1)

Page 3: Carriers (Q4)

On-time and cancellation rate per carrier, a monthly trend per carrier and a scorecard. Source: vw_carrier_monthly.

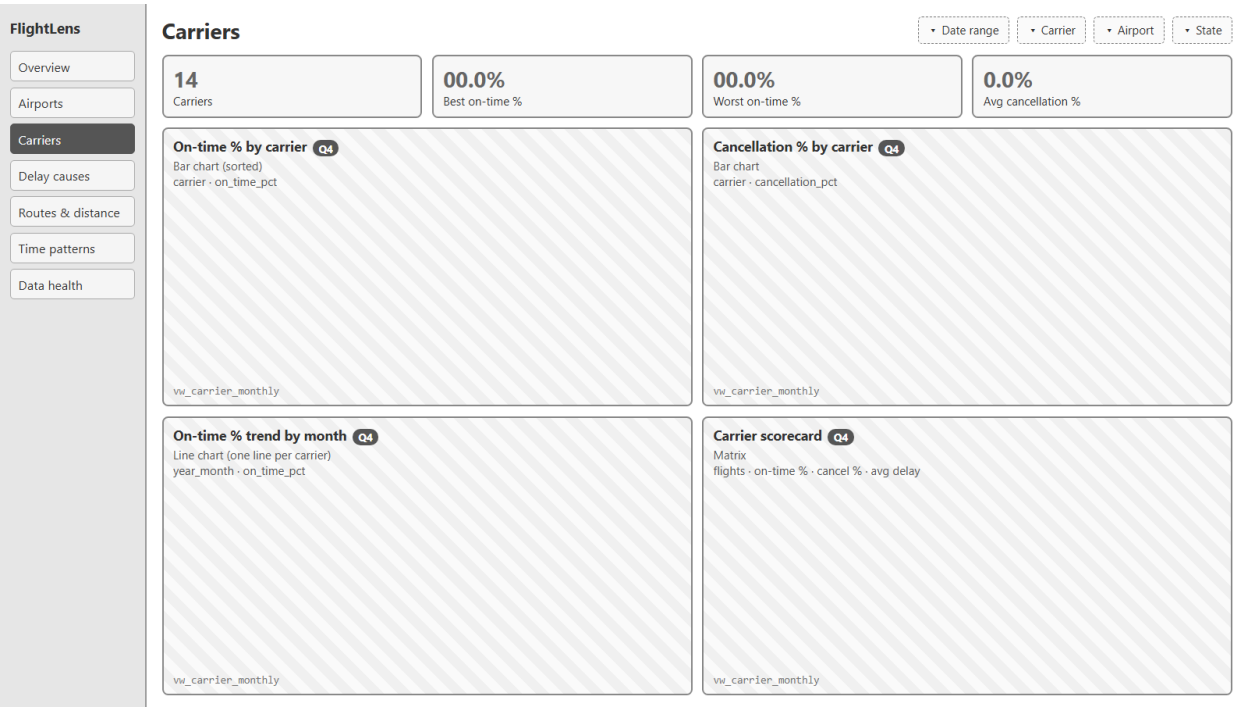


Figure 14. Wireframe: Carriers (Q4)

Page 4: Delay causes (Q2)

Delay minutes by cause, by month, carrier and airport. Source: vw_delay_causes. BTS records cause minutes only for flights that arrived 15 or more minutes late, so these charts explain why late flights were late.

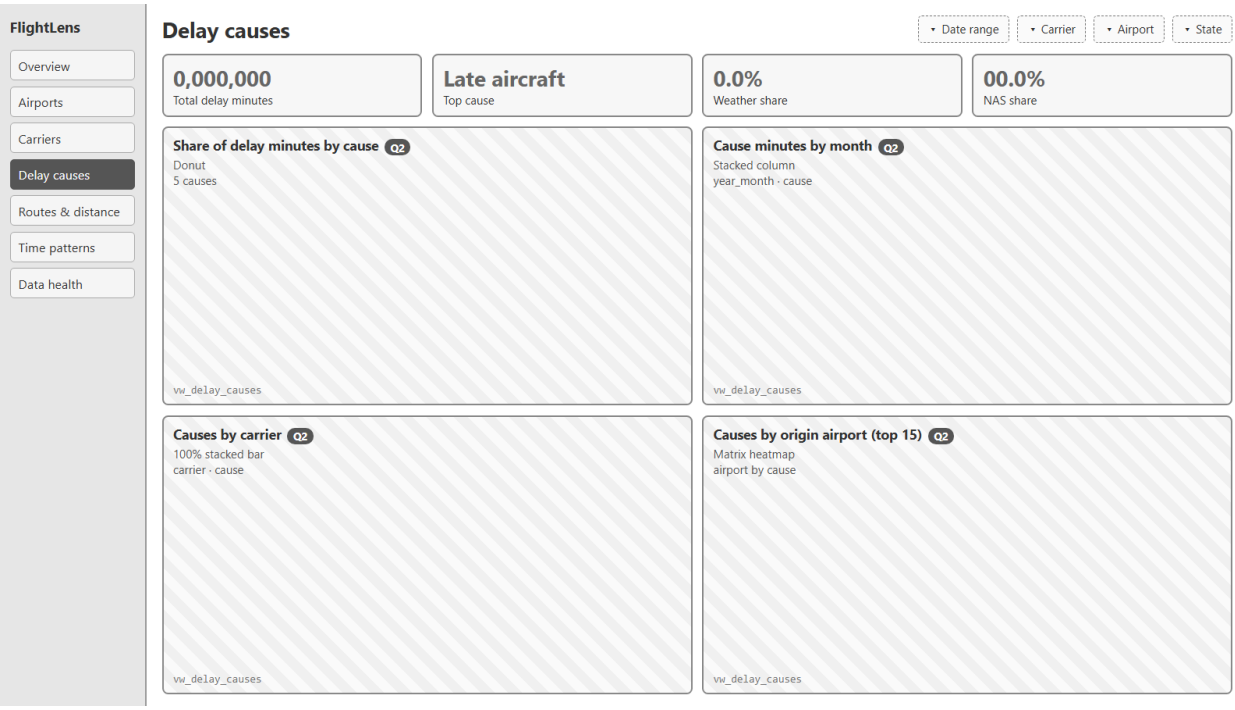


Figure 15. Wireframe: Delay causes (Q2)

Page 5: Routes and distance (Q5, Q6)

The 20 worst routes (minimum 100 flights), delay rate by distance group, distance against delay and routes from a selected origin. Source: vw_route_performance.

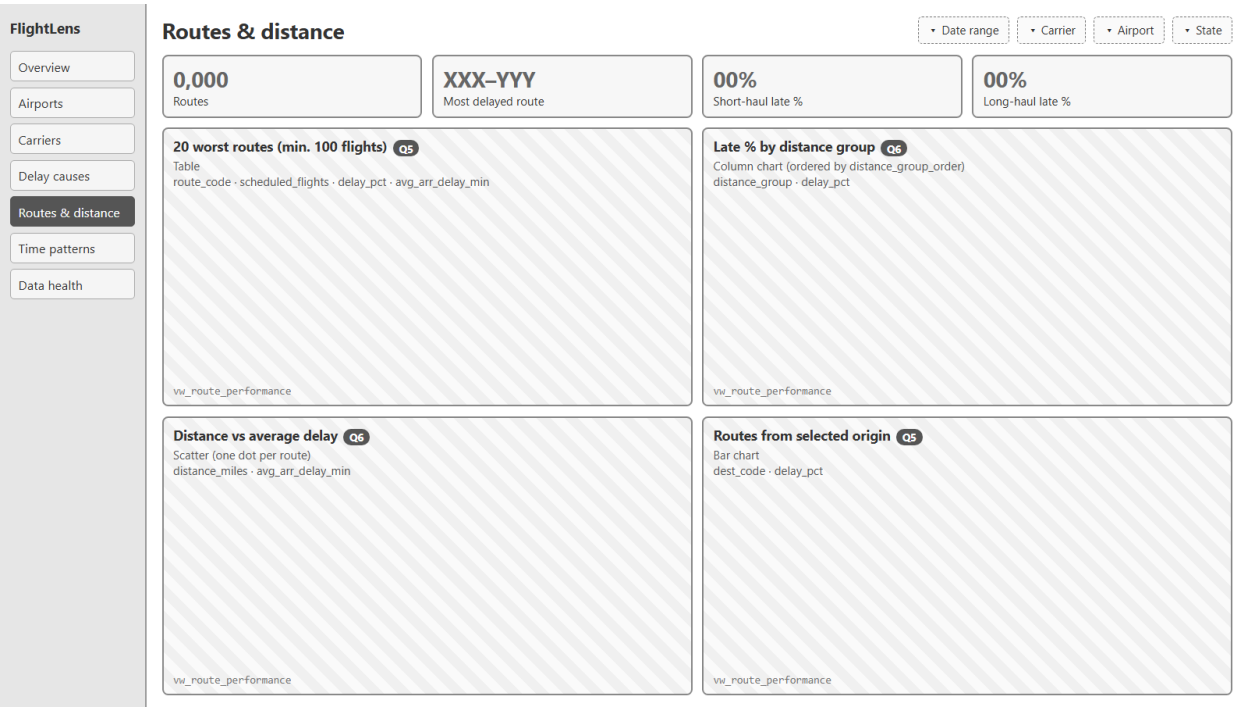


Figure 16. Wireframe: Routes and distance (Q5, Q6)

Page 6: Time patterns (Q3)

Delay rate by month, weekday and departure hour, and the hour by weekday heatmap. Source: vw_flights with dim_date and dim_time.

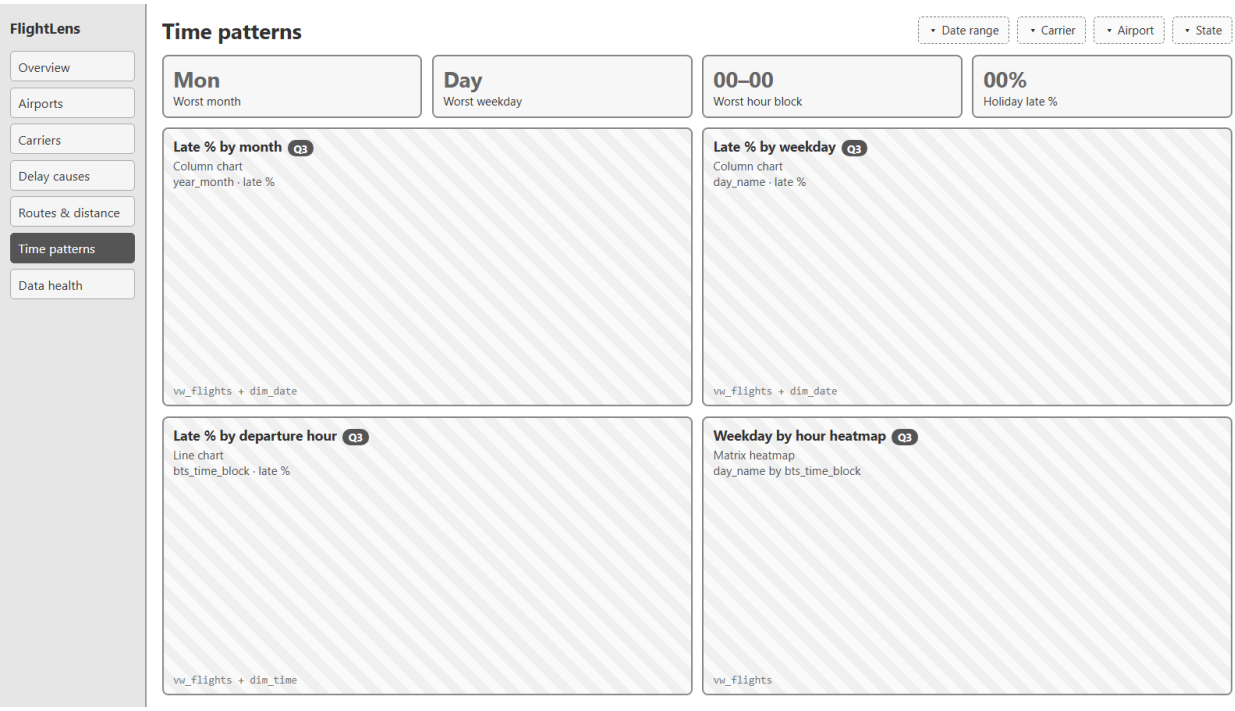


Figure 17. Wireframe: Time patterns (Q3)

Page 7: Data health (FR-11)

Last load, rows loaded per month, quality-check results and quarantined rows. Sources: fact_flights and load_audit.

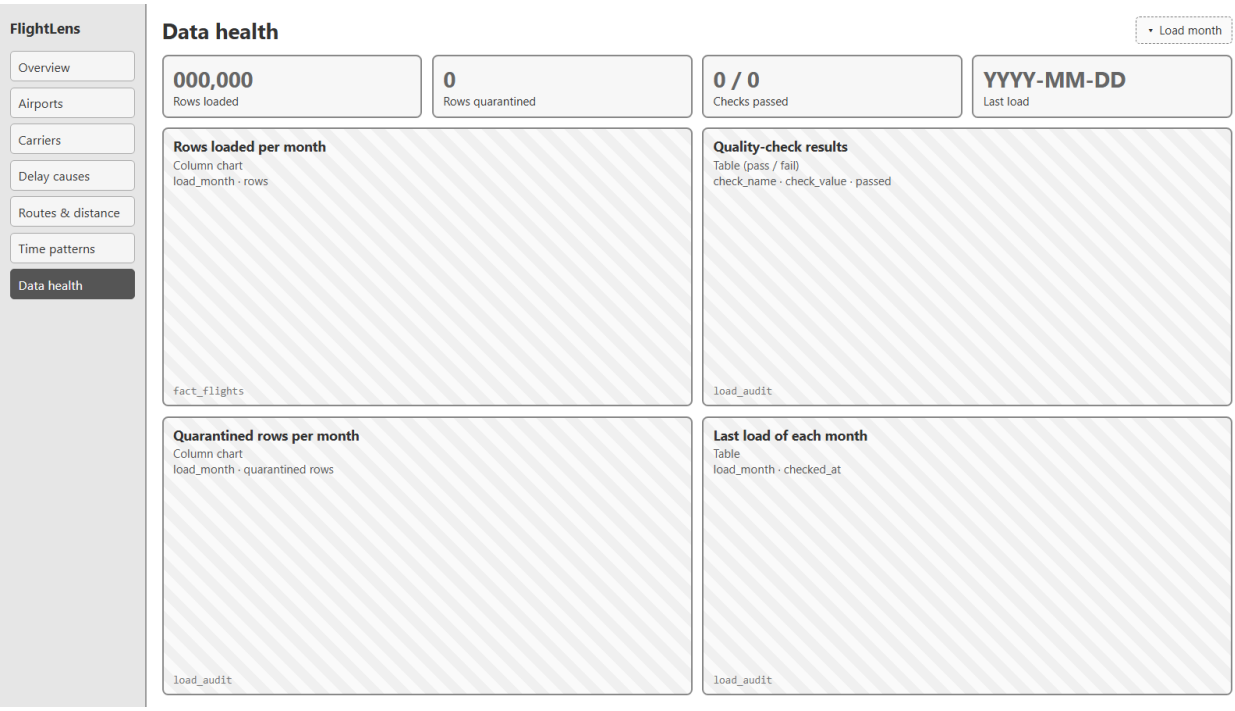


Figure 18. Wireframe: Data health (FR-11)

Page 8: Assistant chat page (Q8)

A separate Streamlit page (UC-05, FR-10). The user types a question or clicks an example. The page shows the answer in plain language, the result table and the SQL used. SQL that is invalid or not a SELECT is not run, and the assistant asks the user to rephrase.

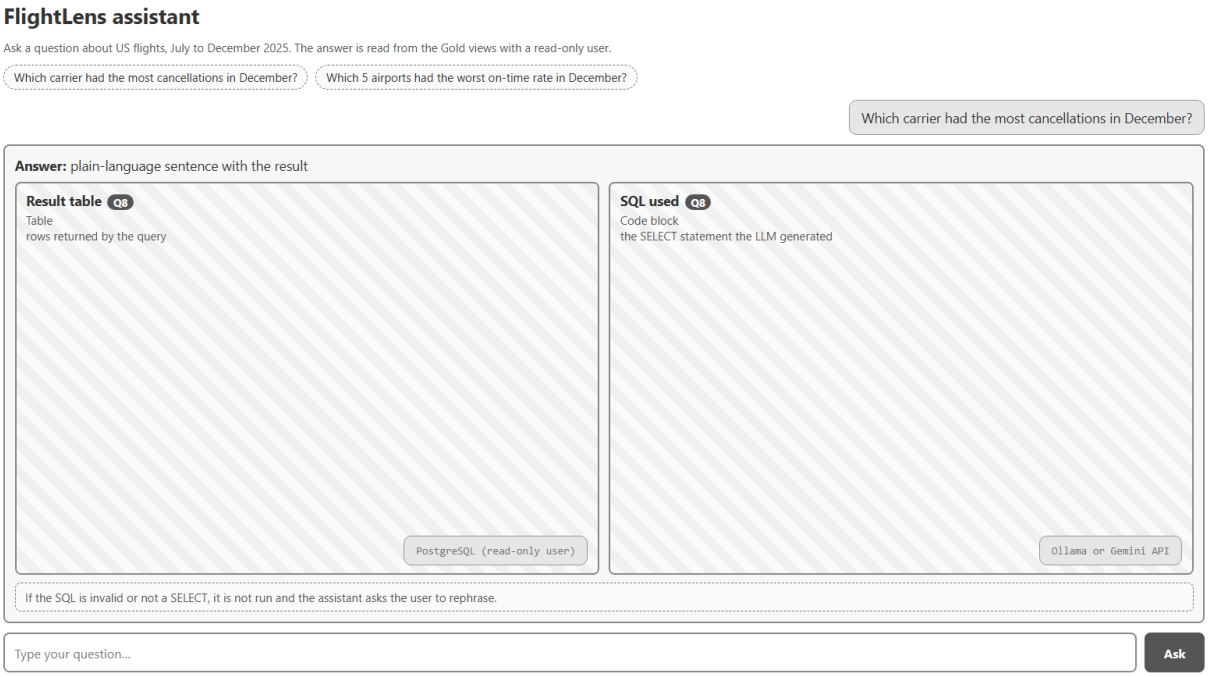


Figure 19. Wireframe: Assistant chat page (Q8)

4.2 UI/UX guidelines

Design principles

- Filters: date range, carrier, airport and state on every dashboard page (FR-08).
- Empty results: when a filter has no data, the page shows "No flights" instead of zeros (UC-03).
- Speed: every page loads in under 5 seconds (NFR-01).
- No training needed (NFR-06): each chart title says what it shows, and the tag on each chart names the business question it answers.
- Same numbers everywhere: every percentage uses the same definition on every page and in the assistant.
- Read-only data: the report and the assistant read the Gold views only, with read-only users (NFR-05).
- Assistant: shows the answer, the result table and the SQL used (FR-10); gives example questions; answers in plain language in under 10 seconds (NFR-01, NFR-06).

Color scheme

The same color always means the same thing on every page. The palette is safe for color-blind users (Okabe-Ito colors), and red and green are never used as a pair.

Use	Color	Hex
Page background / cards	White / light grey	#FFFFFF / #F2F2F2
Text and main values	Dark grey	#333333
On time	Blue	#0072B2
Delayed	Orange	#E69F00
Cancelled	Grey	#7F7F7F
Diverted	Purple	#CC79A7
Cause: Carrier	Dark blue	#0072B2
Cause: Weather	Sky blue	#56B4E9
Cause: National Air System	Green	#009E73
Cause: Security	Purple	#CC79A7
Cause: Late aircraft	Orange	#E69F00
Data health: check failed	Vermillion	#D55E00

Typography

- Font: Segoe UI, the Power BI default, on every page and in the assistant.
- Page title 20 pt bold; chart titles 14 pt semibold; KPI values 24 pt bold with the label under them in 10 pt.
- Axis labels, data labels and table text 10 pt or larger; nothing smaller than 9 pt.
- Numbers use a thousands separator (3,554,943); percentages show one decimal (22.7%); minutes are labeled "min".

Accessibility

- Text contrast of at least 4.5:1 against its background.
- Color is never the only signal: charts have data labels or legends, and the quality-check table shows pass / fail as text.
- Every visual has alt text that says what it shows, for screen readers.
- The tab order on each page follows the reading order: filters, KPI cards, then charts from left to right and top to bottom.
- Tooltips show the exact value and the number of flights behind every percentage.

5. System deployment and integration

5.1 Technology stack

Every component is free and open source, and each one has a direct Microsoft Azure equivalent.

Layer	Technology	Purpose	Azure equivalent
Ingestion	Python (requests, pandas)	Download monthly BTS files	Data Factory copy activity
Orchestration	Apache Airflow	Schedule and monitor the monthly pipeline, retries, alerts	Azure Data Factory
Storage (lake)	MinIO; CSV in Bronze, Delta Lake in Silver and Gold	Bronze, Silver and Gold layers	ADLS Gen2
Processing	PySpark + Delta Lake	Cleaning and transforming 3.55M rows	Azure Databricks
Data quality	Great Expectations	Automated checks between layers	Databricks expectations
Warehouse	PostgreSQL	Star schema and SQL views	Azure Synapse Analytics
BI	Power BI Desktop	Dashboard	Power BI Service
ML	scikit-learn, LightGBM, MLflow, FastAPI	Train, track and serve the delay model	Azure Machine Learning
GenAI	Ollama (local LLM) or Gemini API free tier, Streamlit	Turn plain-English questions into SQL	Azure OpenAI
DevOps	Docker Compose, GitHub, GitHub Actions	One-command setup, version control, CI/CD	Azure DevOps, Container Apps

5.2 Deployment diagram

The whole platform starts with one command, docker compose up, on a team member's laptop. Airflow, Spark, MinIO, PostgreSQL, MLflow, the prediction API and the assistant each run in their own container. Passwords and API keys live in an .env file that is never committed (NFR-05).

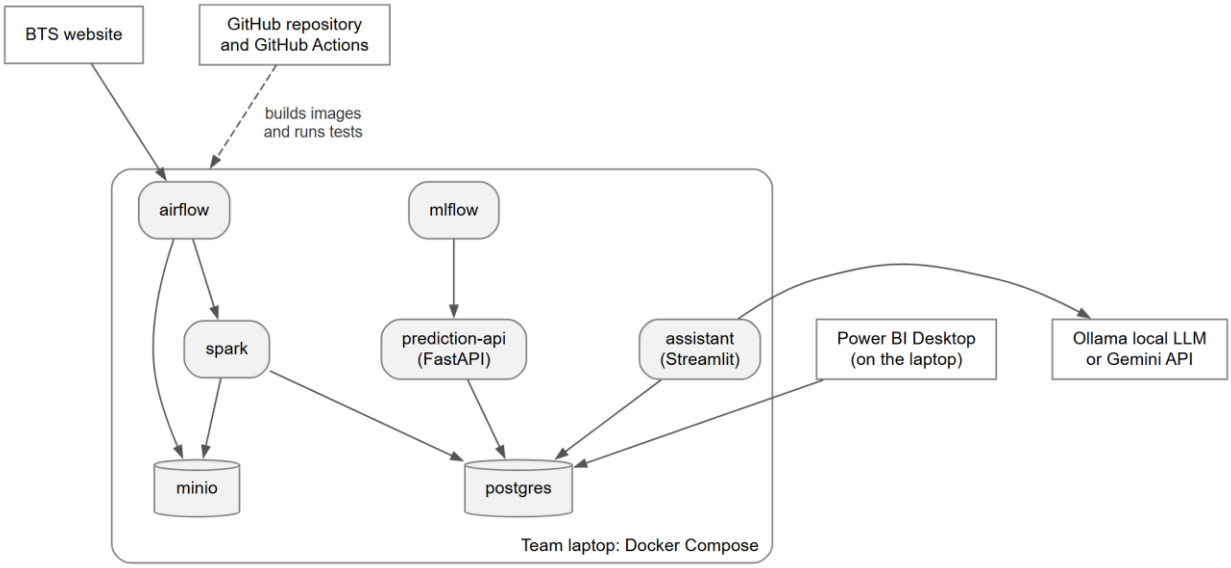


Figure 20. Deployment diagram

5.3 Component diagram

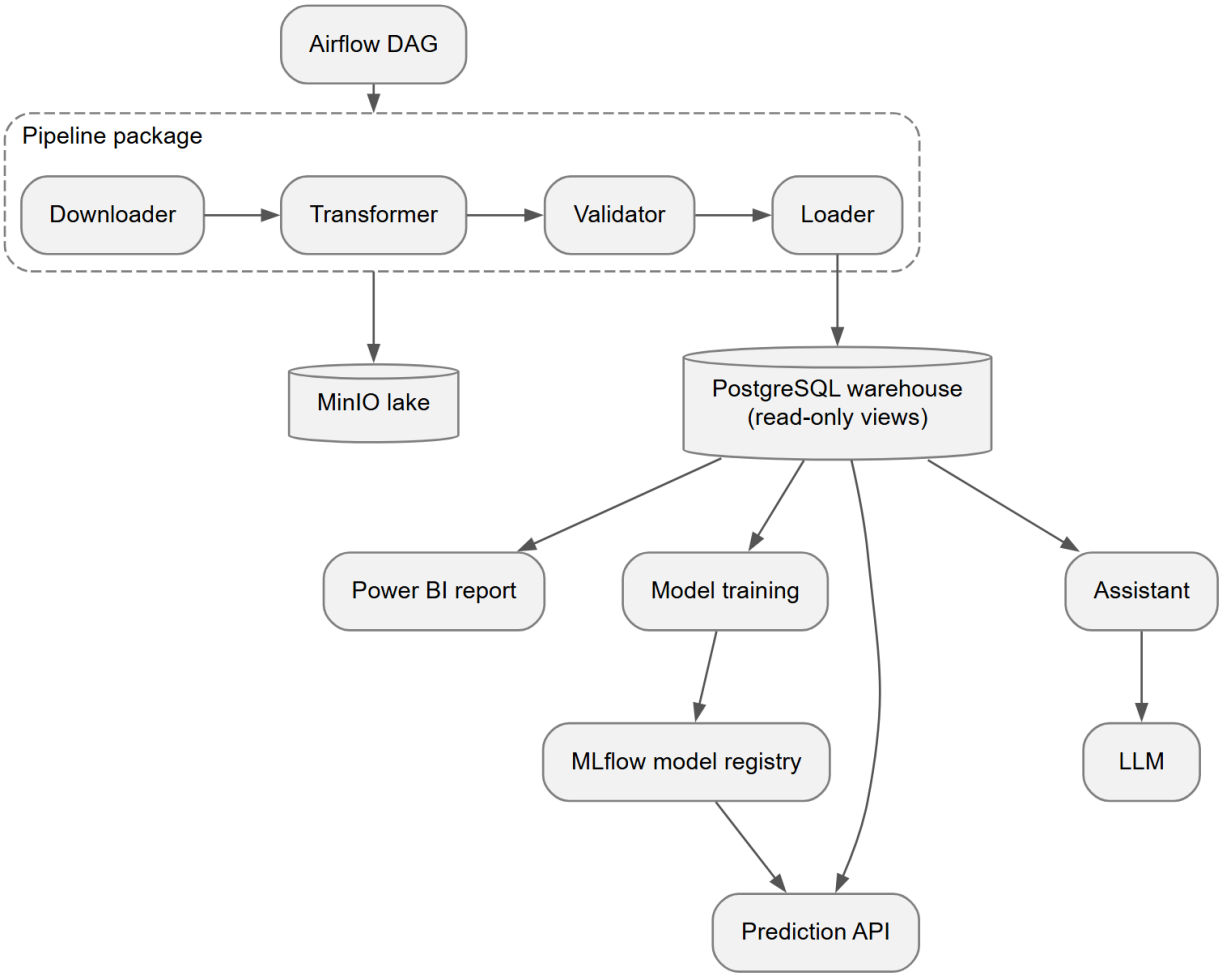


Figure 21. Component diagram

Component	Depends on	Requirement
Airflow DAG	Downloader, Transformer, Validator, Loader	FR-01, NFR-03
Downloader	BTS website, MinIO	FR-01, FR-02
Transformer	MinIO (Bronze to Silver)	FR-03
Validator	MinIO (Silver, quarantine)	FR-04
Loader	MinIO (Gold), PostgreSQL	FR-05, FR-07
Power BI report	PostgreSQL views	FR-08
Model training, MLflow, prediction API	PostgreSQL views	FR-09
Assistant	PostgreSQL (read-only user), LLM	FR-10
Data-health page	PostgreSQL (load_audit)	FR-11

6. Additional deliverables

6.1 API documentation

The delay-prediction API (UC-06, FR-09) is built with FastAPI, which also publishes interactive documentation at /docs. The full API documentation is delivered with the implementation (Phase 5).

Item	Value
Endpoint	POST /predict
Request	carrier, origin, destination, date, scheduled hour
Response	delay probability and an on-time / delayed label
Error	400 with a clear message for an unknown airport or carrier
Target response time	under 500 ms (NFR-01)

The model uses only features known before departure: carrier, origin, destination, distance, scheduled hour, weekday, month, holiday flag, and the route's and carrier's historical on-time rate from earlier months.

6.2 Testing and validation

Level	What is tested
Unit tests	pytest on a small data sample, run by GitHub Actions on every pull request
Data-quality tests	Great Expectations checks between layers; 100% of critical checks pass before Gold (NFR-04)
Integration tests	A full monthly run, and a re-run of the same month gives the same row count with no duplicates (NFR-03)
Model validation	Time-based split: train July–October, validate November, test December 2025; ROC-AUC 0.70 or higher and above the baseline
Assistant test set	30 questions with known answers; 80% or more correct
User acceptance	Each use case, UC-01 to UC-07, run through its main and alternative flow

The detailed test plan, test cases and bug reports are the Phase 6 deliverables.

6.3 Deployment strategy

Item	Plan
Hosting environment	Team laptops with Docker Compose; one command, docker compose up, starts the platform; no cloud account needed
Hardware	At least one laptop with 16 GB RAM runs the full stack; others run only the services they work on
Deployment pipeline	GitHub Actions runs the tests and builds the Docker images on every pull request
Configuration	Passwords and API keys in an .env file that is never committed
Scaling	Monthly partitions, so new months are added without redesign (NFR-02); the Spark jobs can also run on Databricks Free Edition; every component maps to an Azure service for a later move to the cloud
Demo	The final demo runs live on the strongest laptop; a recorded video is the backup